

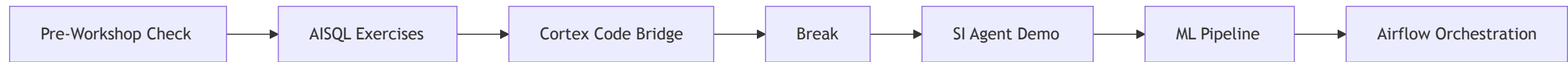
CHOP Workshop

Snowflake AI · Pharmacy Intelligence · ML Pipeline

Audience: CHOP Data Analysts + Data Scientists

Format: 2 hours · Hands-on exercises + Live demos

Workshop Flow — 2 Hours



Time	Block	Format
0:00–0:05	Environment check (Cell 1)	Everyone runs notebook Cell 1
0:05–0:30	AISQL concepts walkthrough	Presenter-led
0:30–1:00	Hands-on AISQL exercises (Cells 2–4)	You run the notebook
0:55–1:00	Cortex Code bridge (Cell 5)	Live demo
1:00–1:10	Break	—
1:10–1:40	SI Agent demo (Cell 6)	Live demo + your questions
1:40–2:00	ML Pipeline + Airflow	Demo + optional hands-on

Before We Begin

What you need to have ready before we start.

Before We Begin: Verify Your Access

Run the readiness check SQL in Snowsight before the workshop starts. All checks should show **PASS**.

Data Analysts (`CHOP_SNOW_INTELLIGENCE`)

Check	What it verifies
A-1	Role is active
A-2	Warehouse active
A-3	Pharmacy scripts view has data
A-4	Drug master view has data
A-5	<code>AI_CLASSIFY</code> is callable
A-6	Prescription entities UDF callable
A-7	Agent visible in Snowsight → Agents

Data Scientists (`ML_ENGINEER`)

Check	What it verifies
B-1	Role is active
B-2	Warehouse active
B-3	Admissions table has data
B-4	Patients table has data
B-5	<code>AI_CLASSIFY</code> is callable
B-6	Agent visible in Snowsight → Agents

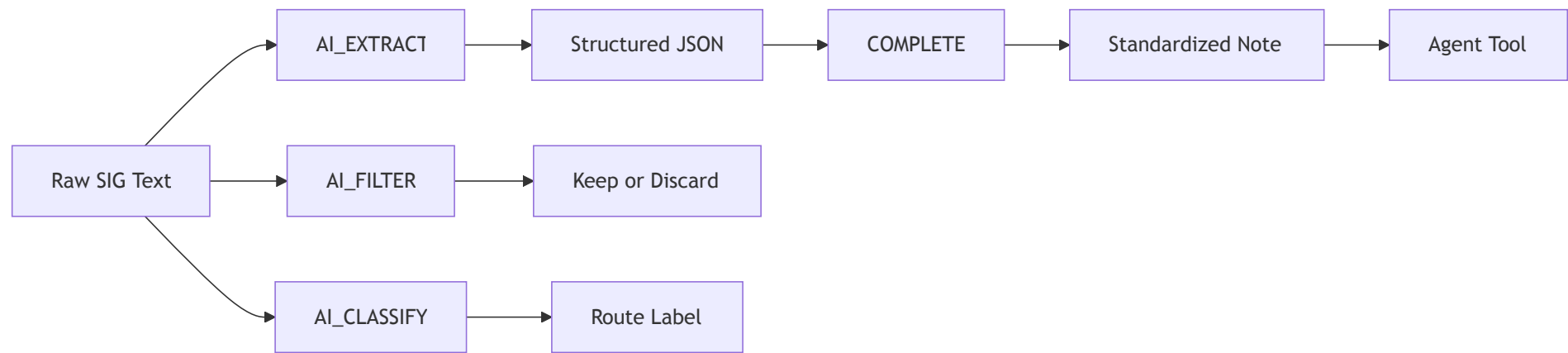
If any check shows **FAIL**, contact the workshop organiser before the session.

AISQL Block

Hands-on Exercises — Cells 2–4 · ~30 min

Both roles participate. Analysts use pharmacy views; Scientists use ML tables.

AISQL: Four Functions, One Flow



Function	What it does	Returns	Analyst use	Scientist use
AI_EXTRACT	Pull structured fields from free text	JSON OBJECT	Extract entities from SIG text	Build narrative from columns → extract
AI_CLASSIFY	Assign text to a controlled label list	JSON OBJECT (:label)	Classify route vs stored ADMINROUTE	Classify discharge → readmission risk tier
AI_FILTER	Boolean quality gate	TRUE / FALSE	Filter incomplete SIG records	Filter low-quality clinical notes
COMPLETE	Free-form LLM transformation	VARCHAR	Standardize informal SIG	Summarize clinical data

All four functions work inside any `SELECT` statement — no extra infrastructure required.

Exercise 1: AI_EXTRACT — Notebook Cell 2 (~10 min)

Part A — Both roles run this (inline, no table access needed):

```
SELECT SNOWFLAKE.CORTEX.AI_EXTRACT(
  'Administer methotrexate 25mg subcutaneously once weekly.
  Monitor LFTs monthly. Hold if WBC < 3.0.',
  ['medication_name', 'dose', 'dose_unit', 'route',
   'frequency', 'monitoring_instructions', 'hold_condition']
) AS extracted_entities;
```

Part B — Analysts (CHOP_SNOW_INTELLIGENCE):

```
SELECT ADMINISTRATIONDIRECTIONS AS raw_sig,
       SNOWFLAKE.CORTEX.AI_EXTRACT(
         ADMINISTRATIONDIRECTIONS,
         ['medication_name', 'dose', 'route',
          'frequency', 'duration']
       )::VARCHAR AS structured_entities
FROM SI_CHOP.CHOP_SNOW_INTELLIGENCE
     .V_PHARMACY_ALLMEDICALSCRIPTS
WHERE ADMINISTRATIONDIRECTIONS IS NOT NULL
LIMIT 3;
```

Part B — Scientists (ML_ENGINEER):

```
SELECT
  'Patient admitted with '||PRIMARY_DIAGNOSIS||
  ', discharged to '||DISCHARGE_DISPOSITION||
  ', LOS '||LENGTH_OF_STAY||' days.' AS narrative,
  SNOWFLAKE.CORTEX.AI_EXTRACT(
    'Patient admitted with '||PRIMARY_DIAGNOSIS||
    ', discharged to '||DISCHARGE_DISPOSITION||
    ', LOS '||LENGTH_OF_STAY||' days.',
    ['primary_condition', 'discharge_setting',
     'readmission_risk_factors']
  )::VARCHAR AS extracted
FROM HEALTHCARE_ML.RAW_DATA.ADMISSIONS LIMIT 3;
```

The EXTRACT_PRESCRIPTION_ENTITIES UDF in the SI agent wraps this exact call — analysts trigger it by asking a question in natural language, no SQL needed.

Exercise 2: AI_CLASSIFY — Notebook Cell 3 (~8 min)

Part A — Both roles (inline):

```
SELECT SNOWFLAKE.CORTEX.AI_CLASSIFY(
    'infuse 500ml normal saline over 4 hours through peripheral IV line',
    ['oral','intravenous','intramuscular','subcutaneous','topical','inhaled','other']
) AS route_classification;
-- Returns: {"label":"intravenous","score":0.99}
```

Part B — Analysts:

```
SELECT
    ADMINISTRATIONDIRECTIONS,
    ADMINROUTE AS stored_route,
    SNOWFLAKE.CORTEX.AI_CLASSIFY(
        ADMINISTRATIONDIRECTIONS,
        ['oral','intravenous','intramuscular',
        'subcutaneous','topical','inhaled','other']
    )::VARCHAR AS ai_route
FROM SI_CHOP.CHOP_SNOW_INTELLIGENCE
.V_PHARMACY_ALLMEDICALSCRIPTS
WHERE ADMINISTRATIONDIRECTIONS IS NOT NULL
LIMIT 5;
-- Where stored_route ≠ ai_route = data quality signal
```

Part B — Scientists:

```
SELECT
    ADMISSION_ID,
    DISCHARGE_DISPOSITION,
    SNOWFLAKE.CORTEX.AI_CLASSIFY(
        'Patient discharged to: ' || DISCHARGE_DISPOSITION,
        ['high_readmission_risk',
        'medium_readmission_risk',
        'low_readmission_risk']
    )::VARCHAR AS ai_risk_tier,
    READMITTED_30D AS actual_outcome
FROM HEALTHCARE_ML.RAW_DATA.ADMISSIONS LIMIT 8;
-- Discussion: LLM tier vs DISPOSITION_RISK_SCORE feature
```

Exercise 3: AI_FILTER + COMPLETE — Notebook Cell 4 (~7 min)

AI_FILTER — quality gate (both roles):

```
SELECT
  SNOWFLAKE.CORTEX.AI_FILTER(
    'take as needed',
    'A complete prescription with dose, frequency, and route'
  ) AS passes_gate_1,      -- FALSE → discard
  SNOWFLAKE.CORTEX.AI_FILTER(
    'Administer 500mg amoxicillin orally every 8 hours for 10 days',
    'A complete prescription with dose, frequency, and route'
  ) AS passes_gate_2;      -- TRUE → keep
```

COMPLETE — standardize an informal note (both roles):

```
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  'Standardize to clinical format with dose, route, frequency, duration, monitoring: '
  || '"give 2 tabs twice a day with food for 2 weeks, watch liver"'
) AS standardized_note;
```

Pipeline pattern: AI_FILTER → AI_EXTRACT on passing records only → store structured output.

Quality gating before extraction keeps your Feature Store and semantic views clean.

Cortex Code Bridge — Build & Deploy Your Agent (~5 min)

Prompt 1 — Generate the spec (*presenter runs live*)

```
I have these objects in
SI_CHOP.CHOP_SNOW_INTELLIGENCE:
- PRESCRIPTION_ORDERS_SV (semantic view)
- MEDICATION_ORDERS_SV (semantic view)
- DRUG_CATALOG_SEARCH (Cortex Search)
- PRESCRIPTION_DIRECTIONS_SEARCH (Cortex Search)
- EXTRACT_PRESCRIPTION_ENTITIES(VARCHAR) (UDF)
- GENERATE_STREAMLIT_APP(VARCHAR) (procedure)
```

```
Generate SQL to CREATE a Cortex Agent
using all 6 as tools for pharmacy questions.
```

The generated `tool_spec` array is structurally identical to the pre-deployed production agent.

Prompt 2 — Deploy your own copy (*everyone runs this*)

```
Take the CREATE AGENT SQL you just generated.
Before running it, make two changes:
```

1. Rename the agent from:
 CHOP_Pharmacy_Intelligence_Agent
to:
 CHOP_Pharmacy_Intelligence_Agent_[ASK ME
 FOR MY SNOWFLAKE USERNAME]
2. Execute the modified SQL in my
 Snowflake account.

```
This keeps my agent separate from the
shared production agent.
```

Each participant ends up with their own agent, e.g.

`CHOP_Pharmacy_Intelligence_Agent_JSMITH`, visible in **Snowsight → Agents**.

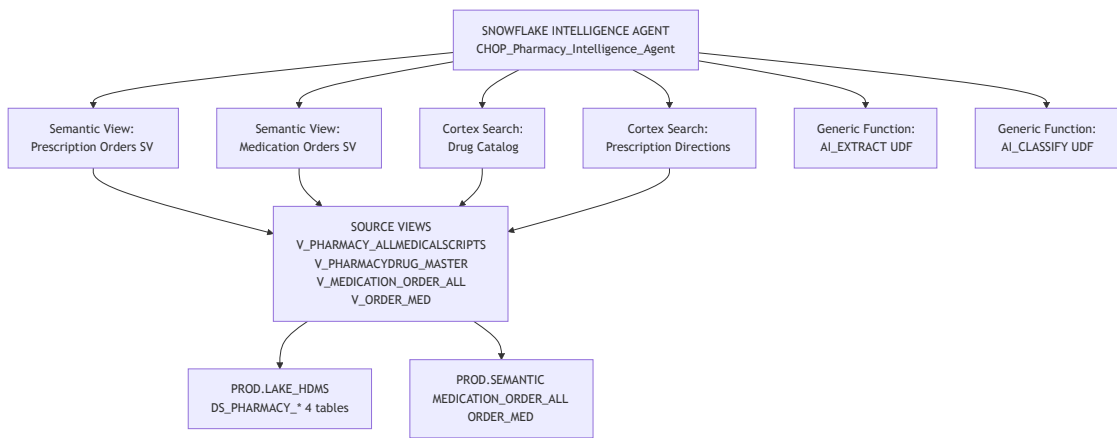


10 minutes

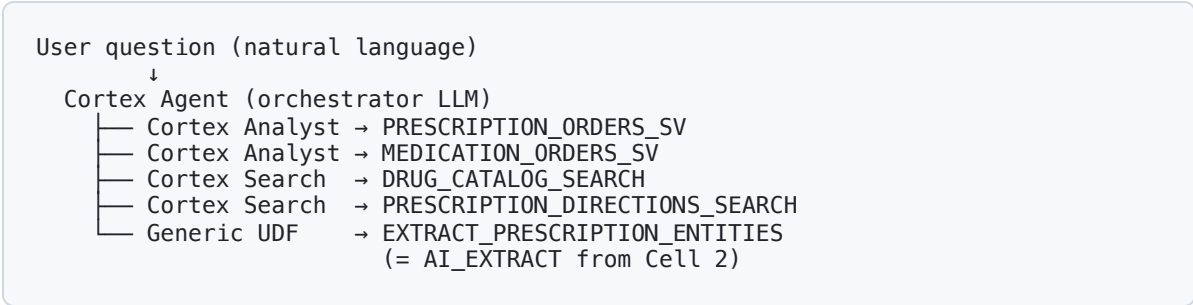
Snowflake Intelligence

Live Agent Demo · ~30 min · Cell 6

SI Architecture — How the Agent Works



Data flow:



Data scope:

Object	Scope
6 source views	Last 12 months of pharmacy data
Search services	Refreshed every 30 days
Agent	Orchestrates all 6 tools automatically
Access	Available via Snowsight → Agents

SI Tools Inventory — 6 Tools

#	Tool Name	Type	Backing Object	When Agent Uses It
1	Query_Prescription_Orders	cortex_analyst_text_to_sql	PRESCRIPTION_ORDERS_SV	Questions about Rx records, costs, dosing, SIG
2	Query_Medication_Orders	cortex_analyst_text_to_sql	MEDICATION_ORDERS_SV	Epic order questions, therapeutic classes
3	Search_Drug_Catalog	cortex_search	DRUG_CATALOG_SEARCH	Find drugs by name, NDC, or category
4	Search_Prescription_Directions	cortex_search	PRESCRIPTION_DIRECTIONS_SEARCH	Find prescriptions by SIG pattern
5	Extract_Prescription_Entities	generic	EXTRACT_PRESCRIPTION_ENTITIES UDF	Extract structured data from SIG free text
6	Generate_Streamlit_App	generic	GENERATE_STREAMLIT_APP procedure	On-demand Streamlit dashboard generation

Open the agent in **Snowsight** → left nav → **Agents** → **CHOP_Pharmacy_Intelligence_Agent**

SI Demo: Try These Questions — Notebook Cell 6

#	Question (type this in the agent chat)	Tool invoked
1	"What are the top 10 most prescribed drugs by script count?"	Cortex Analyst → PRESCRIPTION_ORDERS_SV
2	"Show me all IV-route prescriptions from the last 30 days"	Cortex Analyst → PRESCRIPTION_ORDERS_SV
3	"Find drugs related to amoxicillin in the drug catalog"	Cortex Search → DRUG_CATALOG_SEARCH
4	"Extract medication name and dose from: 'Give 25mg MTX SQ weekly, hold if WBC < 3'"	Generic UDF → EXTRACT_PRESCRIPTION_ENTITIES
5	"What therapeutic classes have the most medication orders?"	Cortex Analyst → MEDICATION_ORDERS_SV

Discussion for Scientists:

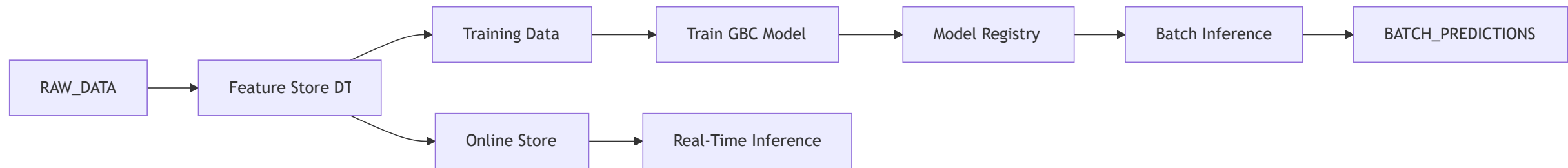
How would you build a readmission risk agent using HEALTHCARE_ML data? What semantic view would you need? What model would you expose as a generic tool?

Expected answer: semantic view on ADMISSIONS + FEATURE_STORE, generic tool calls READMISSION_PREDICTOR model.

ML Pipeline

Git Integration · Feature Store · Model Registry · ~15 min

ML Architecture — End-to-End Pipeline



Stage	Snowflake Object	Details
Raw Data	HEALTHCARE_ML.RAW_DATA	PATIENTS (5K), ADMISSIONS (10.7K), CLINICAL_MEASUREMENTS
Feature Store	PATIENT_CLINICAL_FEATURES\$V1	Dynamic Table, 33 features, 5-min refresh
Online Store	Same Feature View	1-min lag, sub-second lookup for real-time scoring
Model Registry	READMISSION_PREDICTOR V1	GradientBoostingClassifier, scikit-learn, ROC AUC ~0.85
Batch Inference	BATCH_PREDICTIONS	All patients scored via warehouse

Follow along in: `healthcare-readmission-ml/notebooks/00_demo_walkthrough.ipynb` — 35 cells, 8 sections, full end-to-end in one notebook. Run as `ML_ENGINEER` role.

Git Integration — Code Lives in GitHub

The Git repository object (already set up):

```
-- Sync latest code from GitHub:
ALTER GIT REPOSITORY
  HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
  FETCH;

-- Execute a script directly from Git:
EXECUTE IMMEDIATE FROM
  @HEALTHCARE_ML.GIT_INTEGRATION.HEALTHCARE_ML_REPO
  /branches/main/production/run_batch_inference.py;
```

Your code lives in GitHub. Snowflake runs it directly — no copy-paste, no drift.

Scheduled execution:

```
-- Task 1: sync code every 60 min
CREATE OR REPLACE TASK
  HEALTHCARE_ML.TASKS.GIT_FETCH_TASK
  WAREHOUSE = HEALTHCARE_ML_WH
  SCHEDULE = '60 MINUTE'
AS
  ALTER GIT REPOSITORY
    HEALTHCARE_ML.GIT_INTEGRATION
    .HEALTHCARE_ML_REPO FETCH;

-- Task 2: score patients after fetch
CREATE OR REPLACE TASK
  HEALTHCARE_ML.TASKS.BATCH_SCORING_TASK
  WAREHOUSE = HEALTHCARE_ML_WH
  AFTER HEALTHCARE_ML.TASKS.GIT_FETCH_TASK
AS
  EXECUTE IMMEDIATE FROM
    @...HEALTHCARE_ML_REPO
    /branches/main/production
    /run_batch_inference.py;
```

Airflow Orchestration

The same ML pipeline · Full DAG visibility · ~10 min demo

Airflow: What It Adds vs Snowflake Tasks

Snowflake Tasks (already running)

- Simple scheduling (cron / AFTER chain)
- SQL + Python via `EXECUTE IMMEDIATE FROM`
- Visible in `INFORMATION_SCHEMA.TASK_HISTORY`
- Runs entirely inside Snowflake

Apache Airflow (Astro CLI, local)

- **Graph view** — visualize the DAG with dependencies
- **Task logs** — per-task stdout with model metrics, row counts
- **XCom** — pass data between tasks (e.g., ROC AUC from train → register)
- **Retries** with backoff per task
- **80+ provider packages** (Slack alerts, dbt, AWS, etc.)

The same 6-stage pipeline, both ways:

Stage	Snowflake Tasks	Airflow DAG
Validate data	<i>(inline)</i>	<code>validate_data_sources</code>
Check features	<i>(inline)</i>	<code>check_feature_tables</code>
Train model	<code>run_training.py</code>	<code>train_model</code>
Register	<code>run_training.py</code>	<code>register_model</code>
Batch score	<code>run_batch_inference.py</code>	<code>run_batch_inference</code>
Validate output	<i>(inline)</i>	<code>validate_predictions</code>

Both coexist. Snowflake Tasks = lightweight scheduling. Airflow = production MLOps with full observability. The same `src/` Python modules run unchanged in both paths.

Airflow: Live Demo — Watch For

The DAG: `healthcare_ml_pipeline`

6 tasks running sequentially with dependency arrows. Each task calls the same Python modules used by Snowflake Tasks — the code is shared.

In the Airflow UI, watch:

View	What you'll see
Graph view	6-task chain with dependency arrows
<code>train_model</code> logs	ROC AUC score, training progress
<code>register_model</code> logs	Model version registered to Snowflake
<code>run_batch_inference</code> logs	Row count of scored patients
<code>train_model</code> XCom	Model path + ROC AUC passed downstream

Verify the output in Snowflake after the DAG run completes:

```
-- Was the model registered?  
SHOW MODELS IN HEALTHCARE_ML.MODEL_REGISTRY;  
  
-- Were patients scored?  
SELECT COUNT(*) AS scored_patients  
FROM HEALTHCARE_ML.INFERENCE.BATCH_PREDICTIONS;
```

Key takeaway: The DAG gives you a visual audit trail, per-task retry logic, and structured logging — for the exact same workload you already run via Snowflake Tasks.

Summary

What You Leave With Today

AISQL skills:

- Extract structured entities from free text (`AI_EXTRACT`)
- Classify and guardrail data quality (`AI_CLASSIFY` , `AI_FILTER`)
- Standardize and transform text (`COMPLETE`)
- All work inline in any `SELECT` statement

Snowflake Intelligence:

- Natural language → SQL via semantic views
- Agent orchestrates 6 tools automatically
- Ask questions, get answers — no SQL needed

ML Pipeline:

- Feature Store with 33 engineered features
- Model Registry with versioned GBC model
- Git integration: code lives in GitHub, runs in Snowflake
- Scheduled Tasks: auto-fetch + auto-score

Airflow:

- Same pipeline with full DAG visibility
- Per-task logs, XCom data passing, retries
- Production-grade MLOps on top of Snowflake

Resources: Workshop notebooks and SQL scripts at
github.com/sfc-gh-moahmed/Workshops